COMPUTER ENGINEERING

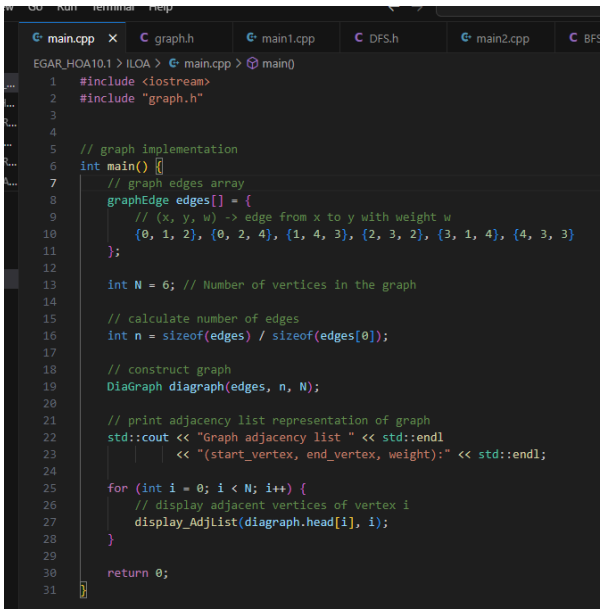
QUEZON CITY

in

COURSE CODE/TITLE:	CPE010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09//1526
NAME:	Jarel Dave R. Egar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ILOA

Main.cpp



TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

Graph.h

EGAR_HOA10.1 > ILOA > graph.h > display_AdjList(adjNode *, int)

```cpp
#ifndef GRAPH_H
#define GRAPH_H

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode **head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node array
        head = new adjNode*[N]();
        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
```

```cpp
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* current = head[i];
            while (current != nullptr) {
                adjNode* next = current->next;
                delete current;
                current = next;
            }
        }
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

#endif
```

OUTPUT

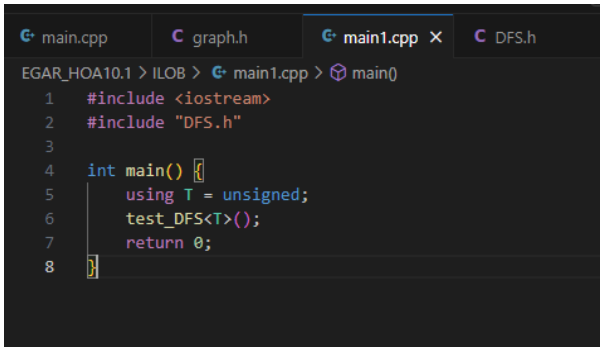
```
PS C:\Users\TIPQC\OneDrive\Documents\CPE2151 - EGAR> cd "c:\Users\TIPQC\OneDrive\Documents\CPE2151 - EGAR\EGAR_HOA10.1\ILOA\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)

PS C:\Users\TIPQC\OneDrive\Documents\CPE2151 - EGAR\EGAR_HOA10.1\ILOA>
```

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

Mainb1.h (ILOB)	
DFS.h	

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

```
#ifndef DFS_H
class Graph {
public:
    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
        } else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph can be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it into the stack.
                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges) {
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order) {
        std::cout << v << std::endl;
    }
}


#endif
```

COMPUTER ENGINEERING

QUEZON CITY

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1 - EGAR> cd "c:\Users\TIPQC\OneDrive\Documents
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1 - EGAR\EGAR_HOA10.1\ILOB>
```

mainb2.cpp

```
main.cpp  graph.h  main1.cpp  BFS.h
EGAR_HOA10.1 > ILOB > main2.cpp > main()
1   #include <iostream>
2   #include "BFS.h"
3
4   int main() {
5       using T = unsigned;
6       test_BFS<T>();
7       return 0;
8   }
9
```

BFS.h

```
EGAR_HOA10.1 > ILOB > BFS.h > ...
1   #ifndef BFS_H
2   #define BFS_H
3
4   #include <iostream>
5   #include <map>
6   #include <queue>
7   #include <set>
8   #include <string>
9   #include <vector>
10
11  template <typename T>
12  class Graph;
13
14  template <typename T>
15  struct Edge {
16      size_t src;
17      size_t dest;
18      T weight;
19
20      inline bool operator<(const Edge<T> &e) const {
21          return this->weight < e.weight;
22      }
23      inline bool operator>(const Edge<T> &e) const {
24          return this->weight > e.weight;
25      }
26  };
27
28  template <typename T>
29  std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
30      for (auto i = 1; i < G.vertices(); i++) {
31          os << i << ":\t";
32          auto edges = G.outgoing_edges(i);
33          for (auto &e : edges) {
34              os << "{" << e.dest << ": " << e.weight << "}, ";
35          }
36          os << std::endl;
37      }
38      return os;
39  }
40
41  template <typename T>
42  class Graph {
43  public:
44      Graph(size_t N) : V(N) {}
45
46      auto vertices() const {
47          return V;
48      }
49
50      auto &edges() const {
51          return edge_list;
52      }
53
54      void add_edge(Edge<T> &&e) {
55          if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
56              edge_list.emplace_back(e);
57          } else {
58              std::cerr << "Vertex out of bounds" << std::endl;
59          }
```

```
        } else {
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges) {
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
```

```
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                queue.push(e.dest);
            }
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order) {
        std::cout << v << std::endl;
    }
}

#endif
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1 - EGAR> cd "c:\Users\TIPQC\OneDrive\Documents\CPE21S1 - EGAR\EGAR_HOA
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1 - EGAR\EGAR_HOA10.1\ILOB>
```

CONCLUSION

This hands-on activity has taught me the difference between a graph and a tree, which is that a tree is a linear type of data structure and a graph is a non-linear type of data structure. In ILO A, it explained the implementation for creating a graph that uses an adjacency list or edge list, and it taught me how to add edges and count the total edges. In ILO B, it taught me the 2 kinds of traversal for graph, Dept-First Search (DFS) and Breadth-First Search (BFS). DFS has a similar traversal with Pre-Order, which starts at the root -> left -> right, but in a graph, since there is no root node, it will then start in the chosen vertex. Lastly is the BFS, which uses the idea of backtracking.

SUPPLEMENTARY

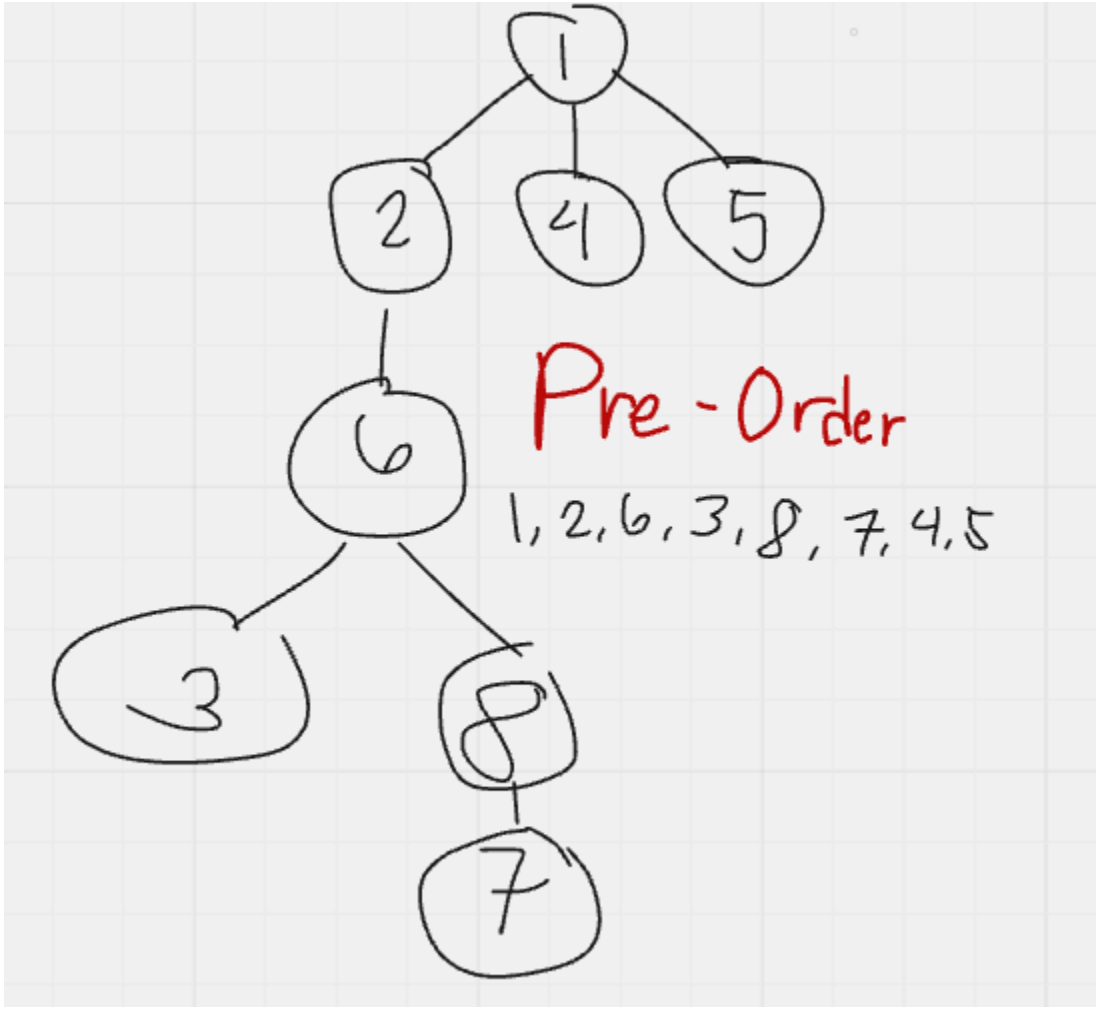
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

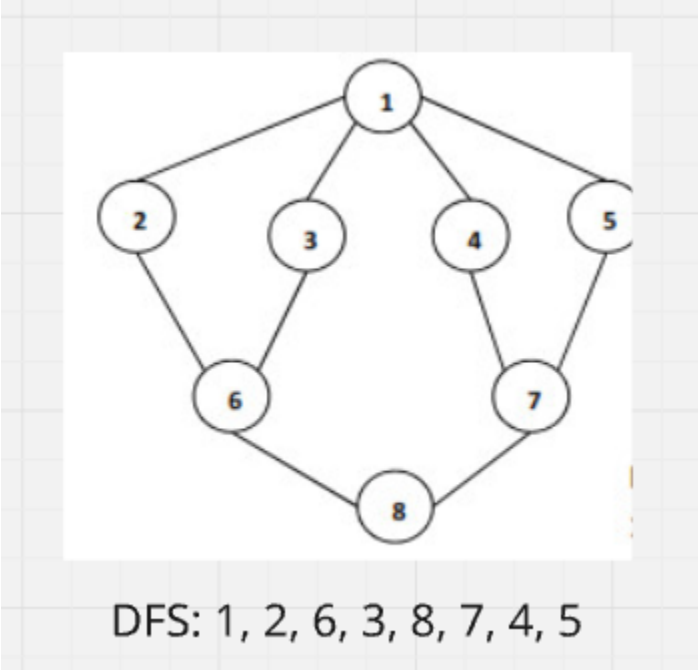
I would recommend using a Depth-First Search (DFS), as this starts at a designated vertex and explores all branches for backtracking, which is what the scenario is requiring the person to do so.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

Graph for Pre-Order Traversal



Graph for DFS



In a pre-order traversal, the pseudocode is like this

START
If node is null:
Return

visit(node)
preorder(node left)
preorder(node right)

END

In a DFS or Depth-First Search

START
Create stack
Push first vertex to stack
Mark first vertex as visited

While stack is not empty
Pop current vertex from stack
Visit current vertex

For each adjacent vertex of current
If adjacent vertex is not visited
Mark adjacent vertex as visited
Push adjacent vertex to stack

END

Sample code for Pre-Order traversal

```cpp
// Pre order
template <typename T>
void BinarySearchTree<T>::preOrder() {
    // Process current node first
    std::cout << key << " ";

    // Traverse left subtree
    if (left != nullptr) {
        left->preOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->preOrder();
    }
}
```

Sample code for DFS

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it into the stack.
                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

The queue structure was used in order to perform a BFS.

4. How many times can a node be visited in the BFS?
ONCE only because this prevents the loop in cyclic or undirected graphs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used my Grammarly extension to fix any grammatical errors.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.